

Machine Learning _Assignment # 3

Instructor: Dr. Madiha Khalid

Program:BS(SE)— Fall 2023

Total Marks: 100

Deadline:May 02, 2026 11:59 PM

Decision Tree & K-Means Clustering

Instructions

- Submitted work is solely your own. Any form of copying or plagiarism will result in zero marks across all involved parties.
- Use only numpy, pandas, matplotlib, and seaborn for data handling and visualization.
- Do not use any built-in sklearn classifiers or clustering models (e.g., DecisionTreeClassifier, KMeans).
- Implement all algorithms from scratch using Python and NumPy.
- Submit a single Colab or Jupyter Notebook (.ipynb) with clear section headings, docstrings on every function, and in-line comments explaining key steps.
- All cells must run top-to-bottom without errors before submission.

Objective

This assignment focuses on two fundamental machine learning techniques. In Part 1, you will build a Decision Tree classifier from scratch using the ID3 algorithm—computing entropy and information gain by hand and in code. In Part 2, you will implement the K-Means clustering algorithm from scratch and apply it to a real-world dataset to discover natural groupings in unlabeled data. Both parts demand full from-scratch implementations; no sklearn estimators allowed.

Task No 01

Dataset

You will use the **Mushroom Dataset** from the UCI Machine Learning Repository. It contains physical descriptions of 8,124 hypothetical mushroom samples, each labeled as either **edible (e)** or **poisonous (p)**. Features include cap shape, cap color, odor, gill size, stalk shape, and several others— all categorical.

Download Link:

<https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.data>

Column names and attribute descriptions are available at:

<https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.names>

Alternatively, download directly in your notebook:

```
import urllib.request

urllib.request.urlretrieve("https://archive.ics.uci.edu/ml/machine-learning-databases/mushroom/agaricus-lepiota.data", 'mushroom.csv')
```

1.1 Dataset Loading and Preprocessing [10 Marks]

a) Load the dataset using pandas. Assign the correct column names (refer to the .names file). Print the first 5 rows, the shape, and the class distribution (how many edible vs. poisonous).

b) Handle missing values: The dataset uses '?' to denote missing entries. Identify which columns contain '?' values and replace them with the column mode (most frequent value).

c) Encode features: Convert all categorical string values to integer codes using a label encoding approach implemented with a Python dictionary— do

not use sklearn's LabelEncoder.

d) Train/Test Split: Manually split the dataset into 80% training and 20% testing using NumPy random shuffling (no sklearn). Report sample counts for each split.

1.2 Manual Entropy and Information Gain Calculation [12 Marks]

Before writing any code, you must carry out the following calculations by hand and include them in a Markdown cell. Show every arithmetic step clearly—no credit is awarded for answers without working.

- **i. Root entropy [4 marks]:** Using only the training split, count the number of edible and poisonous mushrooms. Compute $H(S)$ for the entire training set using the entropy formula above. Round to four decimal places.
- **ii. Information Gain [4 marks]:** The 'odor' attribute has several possible values. For each unique value, compute the proportion and the conditional entropy. Then calculate $IG(S, \text{odor})$. Show the weighted entropy sum step-by-step.
- **iii. Compare two attributes [4 marks]:** Repeat the IG calculation for one other attribute of your choice. State which of the two attributes (odor vs your choice) the ID3 algorithm would select as the root node and explain why in one or two sentences.

1.3 ID3 Decision Tree— From-Scratch Implementation [18 Marks]

Implement the following functions entirely using Python and NumPy. Each function must include a docstring describing its purpose, arguments, and return value.

compute_entropy(y) [3 marks]

Takes a 1-D array of class labels y . Returns the Shannon entropy $H(S)$ computed with the formula above. Handle the edge case where a class proportion is zero (avoid $\log(0)$).

compute_information_gain(X, y, feature_index) [4 marks]

Computes IG for a single feature column. Splits y into subsets based on each unique value of the feature, computes the weighted conditional entropy, and returns $IG(S, A)$.

best_feature(X, y) [3 marks]

Iterates over all feature columns, calls `compute_information_gain`, and returns the index of the feature with the highest information gain.

build_tree(X, y, feature_names, depth=0, max_depth=10) [8 marks]

Recursively constructs the decision tree using ID3. Base cases: (a) all labels are the same—return leaf with that class; (b) no features remain or `max_depth` is reached—return leaf with majority class. Recursive case: find the best feature, split data on its unique values, and recurse on each subset. Represent the tree as a nested Python dictionary.

1.4 Prediction and Evaluation [10 Marks]

- **predict(tree, X_test) [4 marks]:** Write a function that traverses the built tree for each test sample and returns a predicted class label.
- **Accuracy [2 marks]:** Compute classification accuracy on the test set manually as $(\text{correct predictions} / \text{total samples})$. Report the percentage.
- **Confusion Matrix [4 marks]:** Build a 2×2 confusion matrix from scratch by counting TP, TN, FP, FN. Display it as a formatted table and visualize it as a color-coded heatmap using `seaborn` or `matplotlib`. Label axes as 'Actual' and 'Predicted'. Annotate each cell with its count. Briefly comment on what the results tell you about the model's performance.

Hint: Represent the tree as a nested dictionary like `{'feature': 3, 'branches': {0: ..., 1: ..., ...}}` with leaf nodes as plain class labels. Recursion depth is your friend.

Task No 02

Dataset

You will use the **Mall Customer Segmentation Dataset**. It contains 200 records of mall customers with features including **CustomerID**, **Gender**, **Age**, **Annual Income (k\$)**, and **Spending Score (1–100)**. The dataset is well-suited for customer segmentation—a classic real-world clustering task.

Download Link:

<https://www.kaggle.com/datasets/vjchoudhary7/customer-segmentation-tutorial-in-python>

File name to use: Mall_Customers.csv

Alternatively, download directly using kagglehub:

```
import kagglehub
path =
kagglehub.dataset_download('vjchoudhary7/customer-segmentation-tutorial-in-python')
```

You will cluster customers using their **Annual Income** and **Spending Score** features. These two features are chosen deliberately to keep the clusters 2-D and visually interpretable.

2.1 Data Loading and Preprocessing [8 Marks]

- **Load the CSV** using pandas. Print the first 5 rows, dataset shape, and check for missing values.
- **Select features:** Extract only 'Annual Income (k\$)' and 'Spending Score (1-100)' as a NumPy array X of shape (200, 2).
- **Feature scaling:** Apply Z-score standardization (mean = 0, std = 1) to both features

from scratch using NumPy— do not use sklearn's StandardScaler. Store the per-feature mean and std; you will use them to interpret cluster centers later.

- **Scatter plot:** Plot the raw (un-clustered) data as a scatter plot with axes labeled 'Annual income (k\$)' and 'Spending Score'. This will be your baseline visualization.

2.2 K-Means Algorithm— From-Scratch Implementation [22 Marks]

Implement the following functions. Each must have a proper docstring. Do not import or use `sklearn.cluster.KMeans` at any point.

initialize_centroids(X, K, seed=42) [4 marks]

Randomly select K distinct rows of X as initial centroids. Use the provided seed for reproducibility. Return a NumPy array of shape $(K, \text{num_features})$.

assign_clusters(X, centroids) [5 marks]

For each data point in X , compute the Euclidean distance to every centroid and assign the point to the nearest one. Return a 1-D integer array of cluster assignments of length n .

update_centroids(X, labels, K) [5 marks]

For each cluster k , compute the mean of all points currently assigned to it. Return the updated centroid array of shape $(K, \text{num_features})$. Handle the edge case of an empty cluster by keeping the previous centroid unchanged.

compute_distortion(X, labels, centroids) [3 marks]

Compute and return the total distortion J — the sum of squared Euclidean distances from each point to its assigned centroid.

run_kmeans(X, K, max_iters=300, tol=1e-4, seed=42) [5 marks]

Orchestrate the full K-Means loop: initialize, then repeatedly assign and update until centroids move less than tol (use `np.linalg.norm` to measure movement) or max_iters is reached. Record distortion at each iteration. Return final labels, final centroids, and the list of distortion values per iteration.

Run your K-Means with $K = 5$ on the standardized data. Report the final distortion value

and the centroid coordinates (in original scale— reverse the Z-score transformation).

2.3 Choosing K [10 Marks]

Select $K = 5$

1. Run your `run_kmeans` function for $K = 1, 2, 3, \dots, 10$. Use the same seed each time for fairness.
2. Record the final distortion for each value of K .
3. Plot distortion (y-axis) vs. K (x-axis). Label axes, add a title 'Elbow Method for Optimal K ', and mark the suggested elbow point visually (e.g., a vertical dashed line).
4. State in a Markdown cell which value of K you believe is optimal based on your plot and briefly explain your reasoning.

2.4 Visualization and Interpretation [10 Marks]

- **Clustered Scatter Plot [5 marks]:** Using the $K = 5$ result, produce a scatter plot of the original (un-standardized) Annual Income vs. Spending Score. Color-code each point by its cluster

assignment. Overlay the five centroids as large 'X' markers in a contrasting color. Add a legend, axis labels, and a descriptive title.

- **Convergence Plot [3 marks]:** Plot distortion vs. iteration number for $K = 5$, showing how the algorithm converged. The curve should be monotonically non-increasing.

- **Interpretation [2 marks]:** In a Markdown cell, describe each of the 5 clusters in plain English based

on the centroid values (e.g., 'Cluster 3 — high income, low spending score'). What type of mall customer does each cluster likely represent? This should be 5–8 sentences total.

Submission Guidelines

- **File format:** Single .ipynb (Jupyter / Colab) notebook.
- **Naming convention:** RollNumber_Name_A3.ipynb (e.g., BSEF23XXXX_Sara_A3.ipynb)
- **Docstrings:** Every function must have a docstring explaining purpose, inputs, and outputs.
- **No banned imports:** Do not import sklearn, scipy clustering utilities, or statsmodels. NumPy, pandas, matplotlib, seaborn only.
- **Clean execution:** Run all cells top-to-bottom before submission. The notebook must execute without errors.
- **Markdown cells:** Include Markdown cells with section headings for each part and sub-part.

Academic Integrity: Submitted work will be reviewed for plagiarism and AI-generated content. Students may be called for a viva to explain their implementation. Zero tolerance for copying.

Good Luck!

Dr. Madiha Khalid· FCIT· Machine Learning· Spring 2026